Dela Cruz, Cloie Sofia R.
Morales, Rayleen Claire G.
AM1

Support Vector Machine Analysis: Code and Results Discussion

Part 1: Data Exploration

Summary Statistics

```
# This line of code prints a concise summary of the DataFrame, including column
 ↪names, data types, and non-null counts.
# This is important for identifying missing values and ensuring data types
 ↪align with our model's needs.
print(df.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 8 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   Age              1000 non-null   int64
 1   Cholesterol      1000 non-null   float64
 2   SystolicBP       1000 non-null   float64
 3   BMI              1000 non-null   float64
 4   ExerciseHours    1000 non-null   float64
 5   Smoking          1000 non-null   int64
 6   Diabetes         1000 non-null   int64
 7   HeartAttackRisk  1000 non-null   int64
dtypes: float64(4), int64(4)
memory usage: 62.6 KB
None
```

Figure N. Summary Statistics

The code above prints a concise summary of the dataframe that holds our original dataset. The summary statistics reveal 1000 non-null counts among its eight (8) columns. This means **no nulls** are present in the dataset. Furthermore, this gives context on what data types are present for each column, leaving us with four (4) float-type data and four (4) integer-type data.

Descriptive Statistics

```
# This line of code generates a descriptive statistics (mean, std, min, max,␣
 ↪quartiles) for each numeric column.
# This will help us detect outliers, scale differences, and understand central␣
 ↪tendencies across features.
print(df.describe())
```

```
                Age  Cholesterol   SystolicBP          BMI  ExerciseHours  \
count   1000.000000  1000.000000  1000.000000  1000.000000    1000.000000
mean      44.971000   202.682229   120.201911    25.112201       3.928793
std       14.055665    28.696648    14.480513     4.757765       1.917617
min       20.000000   150.000000    90.000000    18.000000       0.000000
25%       35.000000   181.812749   110.280006    21.312898       2.634790
50%       45.000000   201.892314   119.996239    25.000923       3.963516
75%       54.000000   221.866465   129.913730    28.334727       5.278246
max       80.000000   295.793227   178.893566    40.000000      10.000000

           Smoking     Diabetes  HeartAttackRisk
count  1000.000000  1000.000000      1000.000000
mean      0.296000     0.128000         0.153000
std       0.456719     0.334257         0.360168
min       0.000000     0.000000         0.000000
25%       0.000000     0.000000         0.000000
50%       0.000000     0.000000         0.000000
75%       1.000000     0.000000         0.000000
max       1.000000     1.000000         1.000000
```

Figure N. Descriptive Statistics

The code above generates descriptive statistics on the dataset. Below are the gathered column descriptions and insights gathered:

Columns	Description	Insights
Age	• Ranging from 20 to 80 • Normally spread ○ Based on mean ≈ median ≈ 45 • No extreme outliers	• No insights for now
Cholesterol	• Ranging from 150 to 296 • Possibly normally spread ○ Based on mean ≈ median ≈ 202 • Has a moderately large variance ○ Based on the standard deviation = 28.69 ○ Normal for health data • Possibly have outliers	• Outliers suggest high cholesterol and abnormal health conditions

	○ Based on std = 28.69 and max = 296	
SystolicBP	● Ranging from 90 to 179 ● Possibly normally spread ○ Based on the mean ≈ median ≈ 120 ● Has a moderately large variance ○ Based on the standard deviation = 14.48 ○ Normal for health data ● Possibly have outliers ○ Based on std = 14.48 and max = 179	● A healthy average of blood pressure ● Outliers suggest hypertension cases
BMI	● Ranging from 18 to 40 ● Possibly normally spread ○ Based on the mean ≈ median ≈ 25 ● Has a moderately large variance ○ Based on the standard deviation = 4.75 ○ Normal for health data ● No extreme outliers	● Body sizes vary
ExerciseHours	● Ranging from 0 to 10 ● Possibly normally spread ○ Based on the mean ≈ median ≈ 4 ● Has a large variance ○ Based on the standard deviation = 1.91 ○ Variance ratio is around 48% ● No extreme outliers	● Some are sedentary (no exercise) ● Some are active
Smoking	● Mean is ≈ 0.296	● Around 30% of this dataset are smokers
Diabetes	● Mean is ≈ 0.128	● Around 13% of this dataset have diabetes
HeartAttackRisk	● Mean is ≈ 0.153	● Around 15% of this dataset have a high heart attack risk

Target Class Distribution

```
# This line of code uses Seaborn's countplot() to create a bar chart showing␣
 ↪the frequency of each category in the HeartAttackRisk column.
# This will automatically cound how many data points fall into each class and␣
 ↪allow us to check class imbalance.
sns.countplot(x='HeartAttackRisk', data=df)

# This line of code sets the title of the current Matplotlib plot to clearly␣
 ↪describe what's being visualized.
# This helps in the readability and neatness of the code.
plt.title('Class Distribution of Heart Attack Risk')

# This command outputs the plot in our Jupyter Notebook.
# This ensures that the plot is displayed.
plt.show()
```

Figure N. Target Class Distribution Code

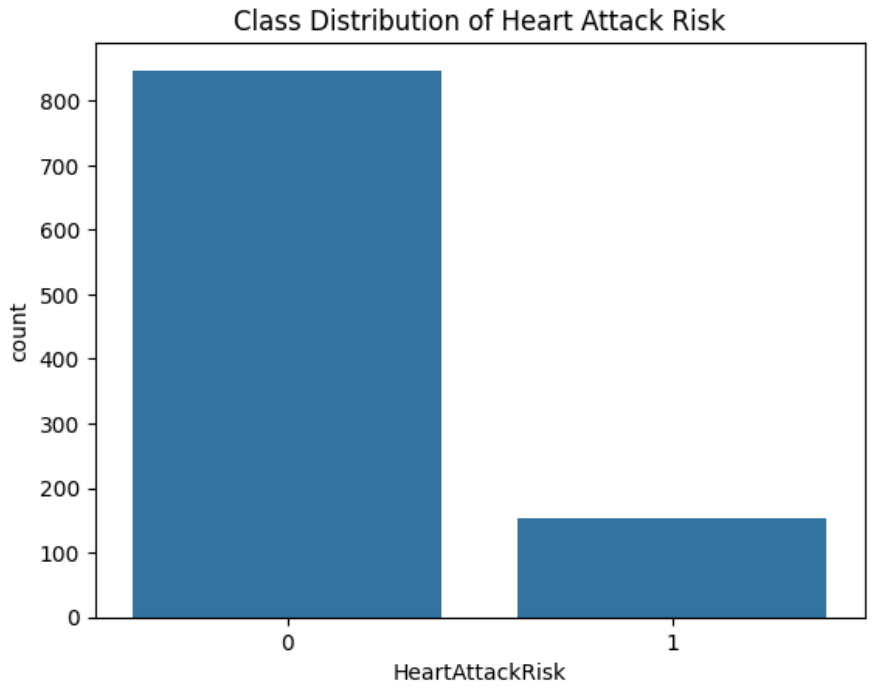


Figure N. Target Class Distribution Output

This code presents the frequency of the target variable, *HeartAttackRisk*. The output presents that the number of patients with high risk is **around 150**, which corresponds to the descriptive statistics above, which indicate that around 15% of the dataset has high risk (15% of 1000). The small proportion of the high risk **indicates a target class imbalance**, wherein the majority of the patients have a low risk (about 85%). This imbalance is significant to avoid bias later during modeling.

Feature Correlation (Features and Target)

```python
# This line of code initializes a new Matplotlib figure with a size of 10 by 8 units.
# This prepares the canvas for plotting before the heatmap is drawn.
plt.figure(figsize=(10,8))

# This line of code creates a heatmap for us to visualize the pairwise correlation between all numerical features.
# The 'annot=True' argument displays the exact correlation coefficients inside each cell.
# Also, the coolwarm colormap visually highlights positive and negative correlations for intuitive analysis.
sns.heatmap(df.corr(), annot=True, cmap='coolwarm')

# This line of code adds a descriptive title to the heatmap.
# This specifically labels the plot as showing a correlation matrix.
plt.title('Correlation Matrix')

# This line of code renders the figure we have constructed so far.
# This finalizes and shows the plot after configuration.
plt.show()
```

```python
# This line of code computes the Pearson correlation of every feature with the target.
# The resut is a ranked list of how strongly each feature correlates with the target.
correlations = df.corr()['HeartAttackRisk'].drop('HeartAttackRisk').sort_values()

# This line of code creates a horizontal bar plot to visualize how each feature
```

```python
# This makes it easier to visually compare the influence of each feature because features with bars further from zero have stronger linear relationships with the target.
correlations.plot(kind='barh', title="Feature Correlation with Target")

# This command displays the generated horizontal bar chart in the jupyter notebook.
# This ensures the plot is rendered after all customization is complete.
plt.show()
```

Figure N. Feature Correlation with Features and Target Code Snippet

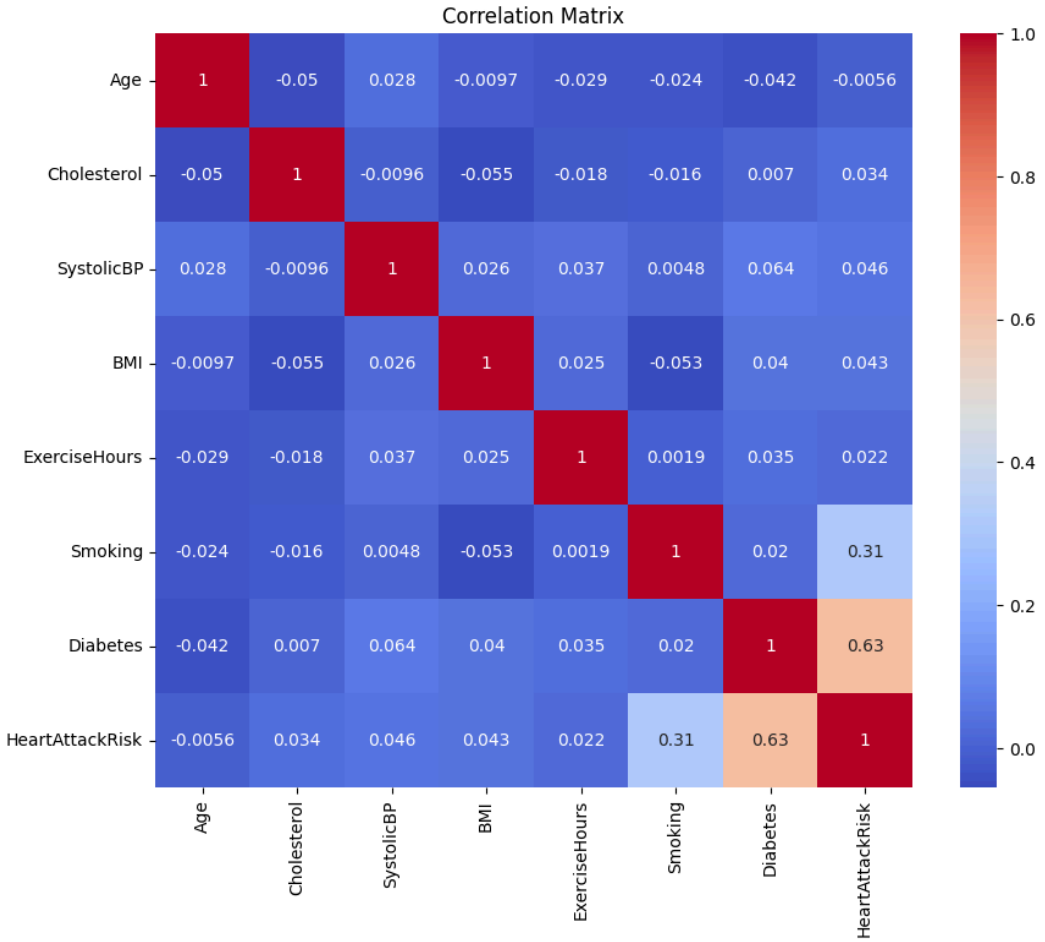


Figure N. Feature Correlation Heatmap

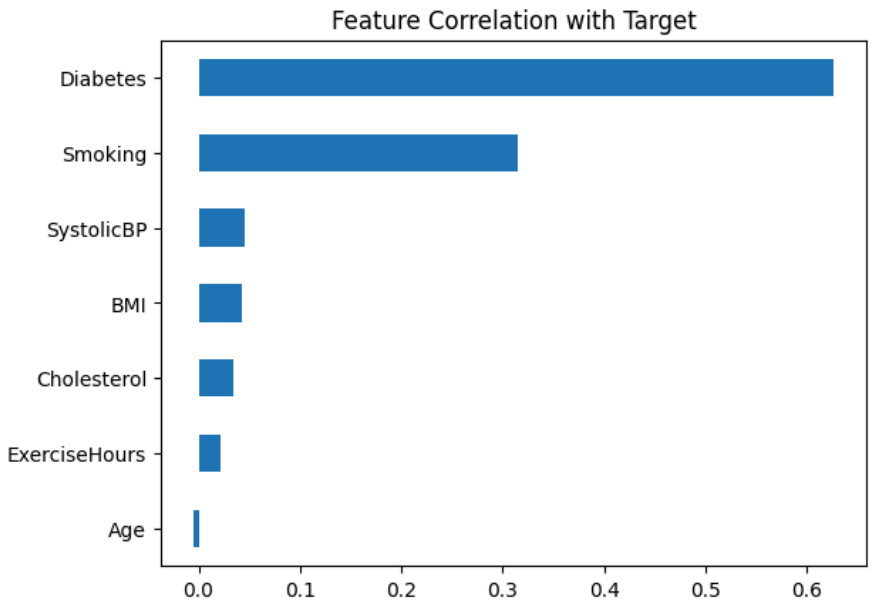


Figure N. Feature Correlation with Target Bar Graph

Feature correlation uses the Pearson pairwise coefficient to associate features and visualize them in a heatmap. The output reveals **no redundant features** among the dataset and a **strong correlation** between *Diabetes* and the target, *HeartAttackRisk,* with a Pearson value of **0.63**. This indicates that diabetes is a strong predictor of heart attack risk. Another significant feature is *Smoking,* with a value of **0.31,** which hints at a moderate correlation with the target. Other features show weak correlations with each other. This indicates that no features are strongly correlated or dependent on each other, including the target. Despite this, it may suggest that **other factors contribute to a higher heart attack risk when combined** with other factors.

The same is done with the bar graph, but in correlation with only the target variable. This corresponds to what has been clear on the heatmap: *Diabetes* has the strongest correlation with the target (around 0.6 on the bar graph), and is followed by *Smoking* (around 0.3 on the bar graph). Other features have correlation values less than 0.1, which indicate weak correlation with the target. Overall, this confirms that **Diabetes and Smoking are the most important features** as predictors of our target.

Anova F-Test

ANOVA F-Test (Feature Scores)

```
# This line creates a new DataFrame X containing all the input (independent)
 features by dropping the target column HeartAttackRisk.
# It ensures that only predictor variables are used for modeling and
 statistical testing, not the target itself.
X = df.drop(columns=['HeartAttackRisk'])

# This line extracts the HeartAttackRisk column from the DataFrame and assigns
 it to y, which represents the target or output variable.
# y must be separated from X for supervised learning workflows and statistical
 tests like ANOVA.
y = df['HeartAttackRisk']

# This line applies standardization to all numeric features in X and stores the
 result in X_scaled.
# The result is a NumPy array where each column (feature) has a mean of 0 and a
 standard deviation of 1, which improves the validity of the F-test.
X_scaled = StandardScaler().fit_transform(X)

# This line performs an ANOVA F-test between each feature in X_scaled and the
 target y, returning arrays of F-scores and corresponding p-values.
# The F-score quantifies how well each feature differentiates between the
 classes in the target variable (higher values = more predictive power).
# The p-values indicate statistical significance; lower values (typically < 0.
 05) suggest that the feature's impact is unlikely due to chance.
f_scores, p_values = f_classif(X_scaled, y)
```

```
# This line of code begins the creation of a new DataFrame anova_df to hold the
 results of the F-test in a tabular format.
# This is important because storing feature names, F-scores, and p-values in a
 DataFrame makes the results easy to explore, sort, and visualize.
anova_df = pd.DataFrame({

    # This line inserts the names of the original input features into the
 Feature column of the anova_df DataFrame.
    # It uses X.columns to retrieve the correct labels from the pre-scaled
 DataFrame so they match the F-scores and p-values.
    'Feature': X.columns,

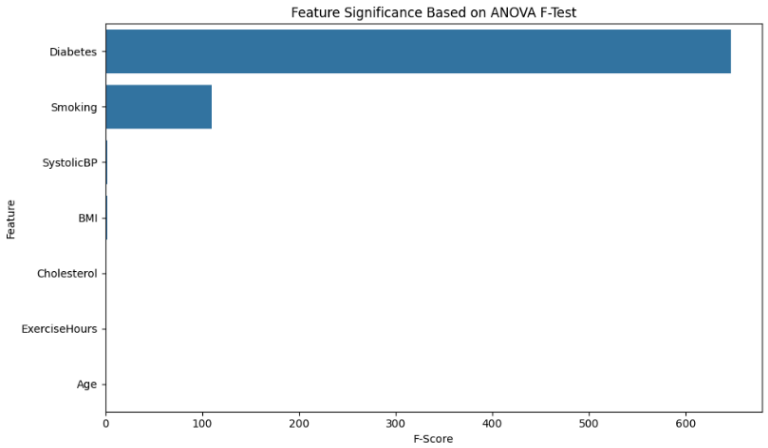
    # This stores the computed F-test statistic for each feature under the
 F-Score column.
    # This metric is directly used to rank feature importance for this
 classification task.
    'F-Score': f_scores,

    # This column stores the statistical significance (p-values) associated
 with each F-score.
    # These values help determine whether the observed feature importance is
 statistically valid or might have occurred by random chance.
    'p-Value': p_values

# After creating the DataFrame, this line sorts it in descending order based on
 F-scores to prioritize the most important features.
# This makes interpretation efficient and focused for reporting or feature
 selection.
}).sort_values(by='F-Score', ascending=False)
```

Figure N. Anova F-Test Code

The ANOVA F-test provides F-scores and p-values to provide more context on the influence of each feature on the target. The F-score quantifies how well each feature differentiates between the classes in the target variable. This means that higher values indicate more predictive power. On the other hand, the p-values indicate statistical significance; lower values (typically < 0.05) suggest that the feature's impact is unlikely due to chance. In performing these features and the target must be identified, with the X variable being scaled for a better test accuracy.



	Feature	F-Score	p-Value
6	Diabetes	646.723372	2.188197e-110
5	Smoking	109.688502	2.005685e-24
2	SystolicBP	2.092837	1.483050e-01
3	BMI	1.840687	1.751772e-01
1	Cholesterol	1.140829	2.857354e-01
4	ExerciseHours	0.476498	4.901733e-01
0	Age	0.031835	8.584258e-01

Figure N. ANOVA F-Test Output

The results of the test still show Diabetes and Smoking as the top features. Diabetes stands out with a high F-score of approximately 647 and a p-value of around $2.19 \times 10^{(-110)}$. This tells us that **diabetes has a very strong statistical association** with heart attack risk. While Smoking follows with an F-score of approximately 110 and a p-value of around $2.19 \times 10^{(-110)}$, which presents **smoking as a statistically significant feature** to the target as well. Other variables have very low F-scores, but p-values higher than 0.05, which tells us that these features are statistically insignificant, but possibly are not strong predictors on its own. All of these also align with the previous exploratory data findings.

Part 2: Data Preprocessing

Feature/Target Split and Feature Scaling

```
# This line of code creates our feature set X by dropping the target column
 ↪'HeartAttackRisk' from the DataFrame.
# It keeps all other columns that we want to use as inputs to our machine
 ↪learning model.
X = df.drop(columns='HeartAttackRisk')

# This line defines our target variable y by selecting the 'HeartAttackRisk'
 ↪column from the original dataset.
# y represents the output label our model will learn to predict based on the
 ↪input features.
# This separation of features and target is essential for fitting any
 ↪supervised machine learning algorithm.
y = df['HeartAttackRisk']
```

```
# This line initializes a StandardScaler object from scikit-learn, which
 ↪standardizes features by removing the mean and scaling to unit variance.
# It prepares the scaler to learn the mean and standard deviation of each
 ↪feature during the fit step.
scaler = StandardScaler()

# This line fits the StandardScaler to the feature data X and then transforms
 ↪it into a scaled version.
# Each feature in X is now rescaled to have a mean of 0 and a standard
 ↪deviation of 1, which ensures fair distance calculations.
# The result X_scaled is a NumPy array that contains the standardized version
 ↪of our feature matrix.
X_scaled = scaler.fit_transform(X)
```

```
# This line of code splits our dataset into training and testing sets using 70/
 ↪30 ratio.
# The random_state=42 ensures reproducibility so we get the same split every
 ↪time we run the code.
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.3,
 ↪stratify=y, random_state=42)
```

Figure N. Feature/Target Split, Feature Scaling, and Train-Test Split

This prepares the data for the model training by splitting the dataframe of the features and the target. To ensure fair distance calculations, features have been standardized after splitting. This is then split again to separate training data from testing data with the ratio of 70/30.

Part 3: Initial SVM Model Implementation

Linear Kernel

```
# This line of code creates an instance of Support Vector Classifier using a linear kernel
# This separates classes with a straight line or hyperplane in higher dimensions.
svm_linear = SVC(kernel='linear')

# This line of code trains the linear SVM model using our training features and labels.
# The algorithm learns a decision boundary that best separates the classes with maximum margin.
svm_linear.fit(X_train, y_train)
```

Polynomial Kernel

```
# This creates a Support Vector Classifier using a polynomial kernal of degree 3.
# This means it will try to fit a cubic deicison boundary to the data.
svm_poly = SVC(kernel='poly', degree=3)

# This trains the polynomial SVM model using our training features and target labels.
# This optimizes the cubic boundary to separate the heart attack risk classes with maximum margin.
svm_poly.fit(X_train, y_train)
```

RBF (Radial Basis Function) Kernel

```
# This line of code initializes a SVC using the RBF kernel, allowing nonlinear decision boundaries.
# The RBF kernels are effective when our data isn't linearly separable.
svm_rbf = SVC(kernel='rbf')

# This line of code trains the RBF-based SVM model on the provided training set.
# The model will determine the best curved boundary that separates high-risk and low-risk individuals.
svm_rbf.fit(X_train, y_train)
```

Figure N. Linear, Polynomial, and RBF Kernel Setup and Training

This code implements three (3) SVM Models that utilizes different kernels: Linear, Radial Basis Function (RBF), and Polynomial. The linear kernel separates classes with a straight line or hyperplane in higher dimensions. The RBF kernel separates classes with nonlinear decision boundaries. The polynomial kernel classifies using a degree parameter, in which our code uses 3. After setting up the kernels, all models were trained with the train data.

Part 4: Initial SVM Model Evaluation

Accuracy, Precision, and Recall

```
# In this part, we define a reusable function neames evaluate_model that takes four parameters: the model's name, the trained model itself, the test features (X_test), and the corresponding labels (y_test).
# We can evaluate a model's performance consistently using this function.
def evaluate_model(name, model, X_test, y_test):

    # This line uses the trained model to generate predictions on the test dataset.
    # These predictions will later be compared to the true values.
    y_pred = model.predict(X_test)

    # This line prints the model's name for visual separation.
    # The ANSI escape code \033[1m starts bold text and \033[0 ends it.
    print(f"\n\033[1m{name}\033[0m")

    # This line prints the accuracy metric.
    # It shows the overall correctness of predictions.
    print("Accuracy:", accuracy_score(y_test, y_pred))

    # This line prints the precision classification metric.
    # It tells us how many predicted positives were correct.
    print("Precision:", precision_score(y_test, y_pred))

    # This line prints the recall metric.
    # It tells us ow many actual positives were correctly predicted.
    print("Recall:", recall_score(y_test, y_pred))

    # This line prints a detailed classification report for each class.
    # This is useful for deeper performance breakdown.
    print("\nClassification Report:\n", classification_report(y_test, y_pred))

    # This generates a visual confusion matrix.
    # This shows true positives, false positives, false negatives, and true negatives.
    disp = ConfusionMatrixDisplay(confusion_matrix(y_test, y_pred))

    # The matrix is color-coded.
    # It uses the "Blues" colormap for clarity.
    disp.plot(cmap="Blues")

    # This line adds a title with the model name.
    # This ensures that we know ehich result the matrix is referring to.
    plt.title(f"Confusion Matrix: {name}")

    # This renders the confusion matrix plot.
    # It finalizes the visualization process.
    plt.show()

# This line calls the evaluate_model() function for the SVM linear model.
# This runs the full evaluation process for this model.
evaluate_model("Linear Kernel", svm_linear, X_test, y_test)

# This line calls the evaluate_model() function for the SVM RBF model.
# This runs the full evaluation process for this model.
evaluate_model("RBF Kernel", svm_rbf, X_test, y_test)

# This line calls the evaluate_model() function for the SVM polynomial model.
# This runs the full evaluation process for this model.
evaluate_model("Polynomial Kernel", svm_poly, X_test, y_test)
```

Figure N. Linear, Polynomial, and RBF Evaluation Code Function

This code creates a function to evaluate the three (3) initially trained models, highlighting the accuracy, precision, and recall values, and including a confusion matrix. The function has been applied to all trained models.

Linear Kernel Evaluation

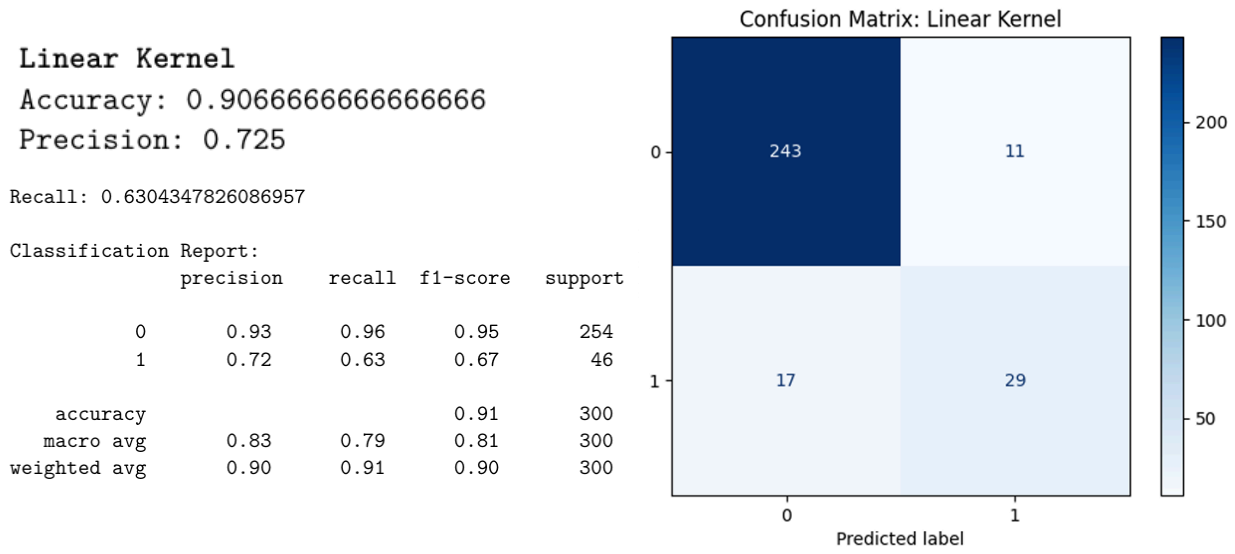


Figure N. Linear Evaluation Output

This Linear Kernel SVM shows an excellent performance on the majority class (low risk) but only moderate recall for the minority class (high risk). While the overall accuracy is high (about 90.67%), the recall of 0.63 for high-risk patients indicates that the **model misses around 37% of actual cases**. This can be a concern in a critical health prediction scenario. Precision for the high-risk class is reasonable (0.72), but **improving recall should be prioritized** — possibly through hyperparameter tuning, class weighting, or resampling techniques.

Radial Basis Function Kernel Evaluation

RBF Kernel
Accuracy: 0.9066666666666666
Precision: 0.725
Recall: 0.6304347826086957

Classification Report:

	precision	recall	f1-score	support
0	0.93	0.96	0.95	254
1	0.72	0.63	0.67	46
accuracy			0.91	300
macro avg	0.83	0.79	0.81	300
weighted avg	0.90	0.91	0.90	300

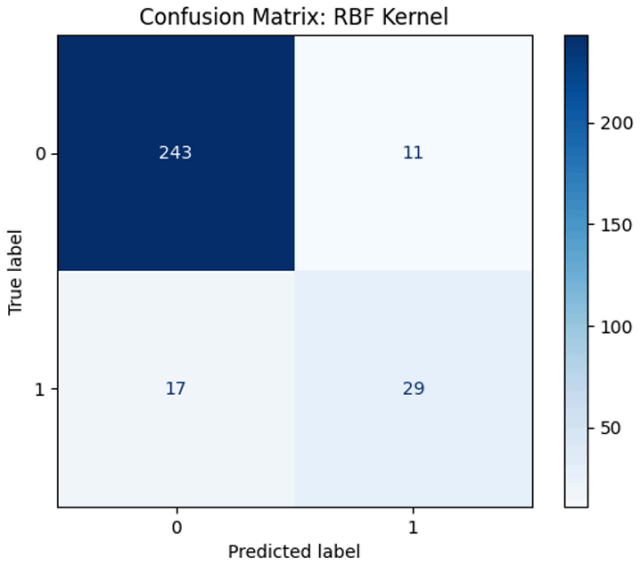


Figure N. RBF Evaluation Output

The RBF Kernel **attained the same performance** as the Linear Kernel, in both accuracy and the classification report. This possibly indicates that the decision boundary for this **dataset is linearly classifiable**, which makes these more complex kernels unnecessary. Because its performance is the same, it also performs well on the majority class, has a moderate recall for the minority class, and misses around 37% of the actual cases.

Polynomial Kernel Evaluation

```
Polynomial Kernel
Accuracy: 0.9033333333333333
Precision: 0.717948717948718
Recall: 0.6086956521739131

Classification Report:
              precision    recall  f1-score   support

           0       0.93      0.96      0.94       254
           1       0.72      0.61      0.66        46

    accuracy                           0.90       300
   macro avg       0.82      0.78      0.80       300
weighted avg       0.90      0.90      0.90       300
```

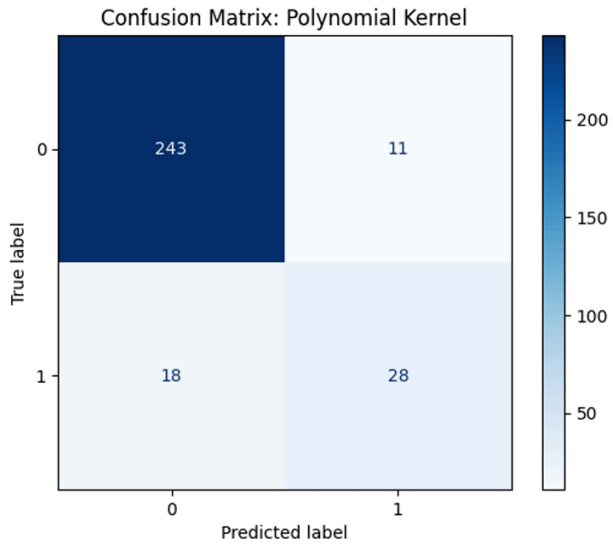


Figure N. Polynomial Evaluation Output

The polynomial kernel has a similar performance to the first two models, with only a **decrease in recall for the high-risk class**. The overall accuracy is still above 90%; however, there are **slightly higher false negatives** compared to the previous ones. This suggests that a **more complex model does not improve detection** of high-risk cases and does not contribute to generalization. Aligned with the RBF performance assumption, increasing model complexity does not help due to the **dataset being linearly classifiable**.

Conclusion

Across the three kernels tested, the **model shows consistent performance** with all the accuracies being higher than 90% and a minor drop in recall and F1-score for the minority class on the polynomial kernel. Insights gathered suggest that **simpler models may handle the dataset** such as the linear kernel SVM.

Part 5: Hyperparameter Tuning

Parameter Grid Setup

```
# This dictionary defines the hyperparameters for the linear SVM Model.
# This will help in conducting the GridSearchCV later.
param_grid_linear = {

    # C controls the regularization strength.
    # This means that lower values increase margin tolerance (so less overfitting), while higher values force the model to classify every training example correctly (risks overfitting) (scikit-learn developers, 2025b).
    'C': [0.01, 0.1, 1, 10, 100],

    # We only specify kernel:['linear'] here to ensure GridSearchCV restricts itself to linear models.
    # This avoid evaluating irrelevant hyperparameters like gamma or degree.
    'kernel': ['linear']
}
```

Figure N. Linear Hyperparameter Grid Code

```
# This dictionary sets the hyperparameter grid for RBF SVM.
# This model can accomodate nonlinear boundaries.
param_grid_rbf = {

    # Again, C controls the regularization strength.
    # We can control it to lessen overfitting.
    'C': [0.01, 0.1, 1, 10, 100],

    # The gamma controls how far the influence of a single training example reaches.
    # Small gamma means far reach (smoother), while large gamma makes it more localized (so complex) (scikit-learn developers, 2025a).
    'gamma': ['scale', 'auto', 0.001, 0.01, 0.1, 1],

    # We only specify rbf here to ensure GridSearchCV restricts itself to RBF kernel.
    # This grid explores combinations of C and gamma values while being restricted to the RBF kernel.
    'kernel': ['rbf']
}
```

Figure N. RBF Hyperparameter Grid Code

```
# This defines the hyperparameters range for polynomial SVMs.
# This allows curved boundaries of varying complexity.
param_grid_poly = {

    # C controls the regularization strength.
    # This allows us to possibly lessen overfitting.
    'C': [0.01, 0.1, 1, 10, 100],

    # The gamma controls how far the influence of a single training example reaches.
    # Small gamma means far reach (smoother), while large gamma makes it more localized (so complex).
    'gamma': ['scale', 'auto', 0.001, 0.01, 0.1, 1],

    # degree controls the polynomial order.
    # This allows the model to try quadratic, cubic, or quartic curves.
    'degree': [2, 3, 4],

    # This grid confines the search to the poly kernel.
    # This ensure maximum relevance.
    'kernel': ['poly']
}
```

Figure N. Polynomial Hyperparameter Grid Output

Dictionaries were set up for holding the hyperparameter grid for all the SVM kernels: *param_grid_linear*, *param_grid_rbf* (for nonlinear boundaries), and *param_grid_poly* (for curved boundaries). These will help in conducting the GridSearchCV later in training an optimized SVM model. Each dictionary holds a ***C* hyperparameter** that **controls the regularization strength** to possibly lessen overfitting through the given wide-ranged list. The RBF and polynomial grid kernels also include a ***gamma* hyperparameter** that **controls how far the influence of a single training example** reaches. The list includes a wide range of potential values that the gamma will be tested on during the GridSearchCV later. Another hyperparameter was added to the polynomial grid kernel, which is the ***degree***. This **controls the polynomial order** of the model, allowing to try quadratic, cubic, or quartic curves.

GridSearchCV Execution

```
# This line of code initializes an empty dictionary to store the best SVM model␣
 ↪for each kernel type.
# Each key will correspond to a kernal and makes it easier to organize and␣
 ↪retrieve the optimized models later.
best_models = {}
```

Figure N. Best SVM Model Dictionary Initialization

```
# This sets up a grid search using 5-fold cross-validation to find the best␣
 ↪linear SVM based on the param_grid_linear values.
# verbose=1 gives progress output during training.
# Grid search is a model hyperparameter optimization technique. It is used to␣
 ↪find the best combination of parameters to improve model accuracy (Nashaat,␣
 ↪2023).
grid_linear = GridSearchCV(SVC(), param_grid_linear, cv=5, scoring='accuracy',␣
 ↪n_jobs=-1, verbose=1)

# This runs the full grid search process on the training set for the linear␣
 ↪kernel.
# Each combination of C in the param grid is evaluated using cross-validation.
grid_linear.fit(X_train, y_train)
```

```
# This prints out the best hyperparameters found by GridSearchCV for the linear␣
 ↪SVM.
# It helps us document and interpret which value of C led to the best␣
 ↪performance.
print("\nBest Linear SVM Params:", grid_linear.best_params_)

# This saves the best trained linear model into the best_models dictionary␣
 ↪under the key 'Linear'.
# best_estimator_ returns the full model trained with the best hyperparameters␣
 ↪on the entire training set (scikit-learn, 2025).
best_models['Linear'] = grid_linear.best_estimator_
```

Figure N. GridSearchCV on Linear Kernel Model

```
# This line of code creates a new GridSearchCV instance to search over the RBF␣
 ↪kernel's parameter space.
# It will iterate through combinations of C and gamma while keeping␣
 ↪kernel='rbf'.
grid_rbf = GridSearchCV(SVC(), param_grid_rbf, cv=5, scoring='accuracy',␣
 ↪n_jobs=-1, verbose=1)

# This performs the hyperparameter search and training for all combinations in␣
 ↪param_grid_rbf.
# After running, it stores the best scoring model internally as best_estimator_.
grid_rbf.fit(X_train, y_train)
```

```
# This displays the best hyperparameters found for the RBF model based on␣
 ↪cross-validation accuracy.
# We'll see which pair of C and gamma performed the best.
print("\nBest RBF SVM Params:", grid_rbf.best_params_)

# Stores the best-performing RBF model into the best_models dictionary under␣
 ↪'RBF'.
# This ensures consistent structure across all three model types.
best_models['RBF'] = grid_rbf.best_estimator_
```

Figure N. GridSearchCV on RBF Kernel Model

```
# Sets up a third GridSearchCV run specifically for polynomial SVMs.
# It will iterate over combinations of C, gamma, and degree.
grid_poly = GridSearchCV(SVC(), param_grid_poly, cv=5, scoring='accuracy',␣
 ↪n_jobs=-1, verbose=1)

# Executes the hyperparameter search and training for the polynomial kernel.
# Each model is trained and validated 5 times per configuration.
grid_poly.fit(X_train, y_train)
```

```
# Prints the best C, gamma, and degree values found for the polynomial kernel.
# This gives insight into what level of model complexity worked best.
print("\nBest Polynomial SVM Params:", grid_poly.best_params_)

# Saves the trained best polynomial SVM model into the dictionary under␣
 ↪'Polynomial'.
# This finalizes the process of optimizing all three kernels.
best_models['Polynomial'] = grid_poly.best_estimator_
```

Figure N. GridSearchCV on Polynomial Kernel Model

An **empty dictionary,** *best_models,* was primarily initialized to hold the best SVM model for each kernel type. For this to work, a key will correspond to a kernel for easier access later. Each kernel also had a setup for a **grid search using 5-fold cross-validation** to find the kernel SVM based on the parameter grid setup earlier, namely, *grid_linear*, *grid_rbf*, *grid_poly*. Grid search is used in this activity as a hyperparameter optimization technique to improve model accuracy. **Each grid is then trained with the training data** that runs the full grid search process, in which each combination of C in the param grid is evaluated using cross-validation. The best-trained kernel model is saved in the dictionary, with its kernel type as its key.

```
Fitting 5 folds for each of 5 candidates, totalling 25 fits

Best Linear SVM Params: {'C': 0.01, 'kernel': 'linear'}
```

Figure N. Best Linear SVM Hyperparameter

The linear grid search was performed over five (5) different values of the regularization parameter (C), with each being evaluated with the 5-fold cross-validation, which totaled to 25 model fits. The optimal configuration selected was **C = 0.01** This indicates that **stronger regularization leads to better performance** on this dataset, suggesting a **simpler and well-regularized model is enough** to capture the patterns in the data without the risk of overfitting.

```
Fitting 5 folds for each of 30 candidates, totalling 150 fits

Best RBF SVM Params: {'C': 1, 'gamma': 0.01, 'kernel': 'rbf'}
```

Figure N. Best RBF SVM Hyperparameter

The RBF grid search was performed over 30 parameter combinations with 5-fold cross-validation, which totaled 150 model fits. The optimal configuration selected was **C = 1** with a **gamma = 0.01**. This indicates that **moderate regularization and a low gamma produced the best performance**, suggesting a smooth decision boundary was utilized to avoid overfitting and underfitting. These results correspond with the earlier insights that increasing the model complexity and flexibility does not produce better results on this dataset.

```
Fitting 5 folds for each of 90 candidates, totalling 450 fits

Best Polynomial SVM Params: {'C': 1, 'degree': 3, 'gamma': 'scale', 'kernel':
'poly'}
```

Figure N. Best Polynomial SVM Hyperparameter

The polynomial grid search was performed over 90 candidate combinations that resulted in 450 fits. The optimal configuration selected was a polynomial **degree = 3**, with **C = 1** and **scaled gamma**. This indicates that a **third-degree polynomial with a moderate regularization and a default scaled gamma produced the best performance** that avoided overfitting and underfitting. Despite this complex and flexible model, the evaluation shows similar performance to the linear and RBF kernels, implying that linear provides the best benefit for the dataset.

Part 6: Optimized SVM Model Implementation

Retrieve and Assign Each Optimized Model

```
# This retrieves the best-performing linear SVM model stored under the 'Linear' key in the best_models dictionary.
# It was previously selected by GridSearchCV above.
svm_linear_optimized = best_models['Linear']

# This retrieves the best-performing RBF SVM model from the same dictionary.
# It was tuned specifically for the 'rbf' kernel, considering both C and gamma.
svm_rbf_optimized = best_models['RBF']

# This accesses the best polynomial SVM that was optimized over C, gamma, and degree.
# It's now stored in svm_poly_optimized for clarity and usability.
svm_poly_optimized = best_models['Polynomial']
```

Figure N. Best Polynomial SVM Hyperparameter

This code helps prepare the trained kernel models by accessing and retrieving the best-performing kernel SVM model from the *best_models* dictionary. It will then be assigned to individual variables: *svm_linear_optimized*, *svm_rbf_optimized*, and *svm_poly_optimized,* that will be used later for model evaluation.

Part 7: Optimized SVM Model Evaluation

Performance Metrics

```
# This line of code calls our evaluate_model function to test the performance of the optimized linear SVM on the test set.
# It prints classification metrics like accuracy, precision, recall, F1-score, and plots a confusion matrix for visual interpretation.
evaluate_model("Optimized Linear SVM", svm_linear_optimized, X_test, y_test)
```

```
# This line evaluates the optimized RBF SVM, applying the same standardized performance checks.
# The test results reflect how well the radial model generalizes after being tuned on parameters like C and gamma.
evaluate_model("Optimized RBF SVM", svm_rbf_optimized, X_test, y_test)
```

```
# Here, we evaluate the optimized polynomial SVM, which is more complex due to its use of degree and gamma.
# Labeling it as "Optimized Polynomial SVM" ensures clarity in reports, plots, and comparison tables.
evaluate_model("Optimized Polynomial SVM", svm_poly_optimized, X_test, y_test)
```

Figure N. Optimized Linear, RBF, and Polynomial Evaluation Code

This code utilizes the earlier defined function that evaluates models with accuracy, precision, and recall values, and including a confusion matrix. The function has been applies to all optimized trained models.

Optimized Linear Kernel Evaluation

```
Optimized Linear SVM
Accuracy: 0.9066666666666666
Precision: 0.725
Recall: 0.6304347826086957

Classification Report:
              precision    recall  f1-score   support

           0       0.93      0.96      0.95       254
           1       0.72      0.63      0.67        46

    accuracy                           0.91       300
   macro avg       0.83      0.79      0.81       300
weighted avg       0.90      0.91      0.90       300
```

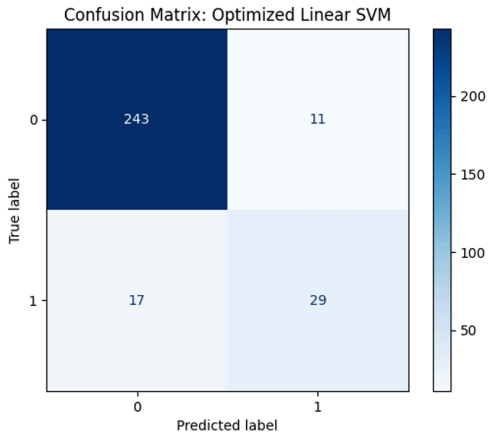


Figure N. Optimized Linear Kernel Evaluation

The optimized linear SVM with the best parameters from hyperparameter tuning achieved an **overall strong performance** with about **91% accuracy**, with an excellent detection of the low-risk class. However, while the **precision value for high-risk patients** is acceptable **(72%),** the **recall value of 63%** raises the concern of **actual high-risk cases mismatch**. This is crucial considering this concerns the health risk of an individual.

In comparison with our initial linear SVM model, the optimization did not significantly change performance. This show that the previous Linear SVM was near optimal, and that this dataset truly is better with linear classification. Despite being the well-suited model, the evaluation reveals that **further steps may be needed** to capture more high-risk patients.

Optimized RBF Kernel Evaluation

```
Optimized RBF SVM
Accuracy: 0.9066666666666666
Precision: 0.725
Recall: 0.6304347826086957

Classification Report:
              precision    recall  f1-score   support

           0       0.93      0.96      0.95       254
           1       0.72      0.63      0.67        46

    accuracy                           0.91       300
   macro avg       0.83      0.79      0.81       300
weighted avg       0.90      0.91      0.90       300
```

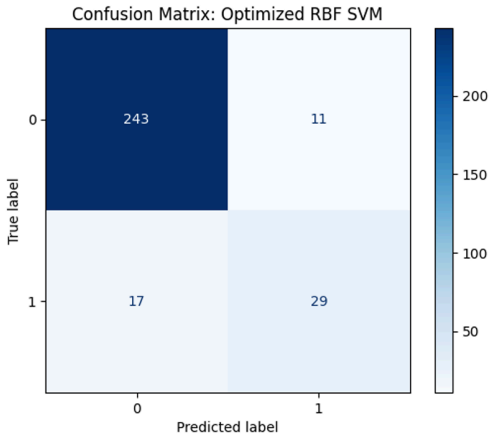


Figure N. Optimized RBF Kernel Evaluation

The optimized RBF Kernel SVM's ability to model nonlinear relationships still **resulted in similar results compared to the Linear SVM**. This result highlights the possibility that the **dataset does not require nonlinear decision boundaries** and that the simpler linear models are sufficient. In terms of switching kernels, it would be better to **address class imbalances for a greater impact** on performance improvement.

Optimized Polynomial Kernel Evaluation

```
Optimized Polynomial SVM
Accuracy: 0.9033333333333333
Precision: 0.717948717948718
Recall: 0.6086956521739131

Classification Report:
              precision    recall  f1-score   support

           0       0.93      0.96      0.94       254
           1       0.72      0.61      0.66        46

    accuracy                           0.90       300
   macro avg       0.82      0.78      0.80       300
weighted avg       0.90      0.90      0.90       300
```

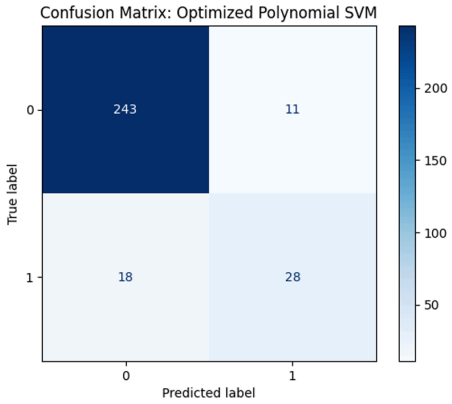


Figure N. Optimized Polynomial Kernel Evaluation

The optimized Polynomial SVM showed a **weaker performance** compared to the linear and RBF kernels. This is highlighted with the **slight drop in recall and F1-score for the high-risk class**. This corresponds with the earlier insights that a more complex polynomial decision boundary does not improve the model performance for the dataset.

Part 8: Model Output Visualization

Dimensionality Reduction (PCA)

```
# This function will be used to visualize the decision boundary of our SVM models.
# It will help us understand how well the model separates classes in 2D space after PCA.
def visualize_svm_decision_boundary(model: SVC, model_name: str, X_train, X_test, y_train, y_test):

    # This line initializes a PCA object to reduce features to 2D.
    # Reducing to 2D is necessary for plotting on a 2D graph while preserving variance (GeeksForGeeks, 2025).
    pca = PCA(n_components=2)

    # This line fits PCA to the training set and transforms it into a 2D representation.
    # The result is a compressed version of X_train containing only two principal components.
    X_train_2D = pca.fit_transform(X_train)

    # This line code transforms the test data using the already-fitted PCA model from training.
    # This ensures the test set is projected into the same 2D space as the training set.
    X_test_2D = pca.transform(X_test)

    # This line of code creates a new SVM classifier.
    # It uses the same hyperparameters as the original model.
    model_2D = SVC(

        # This line ensures that the kernel matches the stored kernel.
        # Its values will be linear, rbf, and polynomial.
        kernel=model.kernel,

        # This line ensures that the SVM model will use consistent C values.
        # These C values will be sourced from the given model.
        C=model.C,

        # This line of code uses conditional logic.
        # The gamma and degree will only be included when required by the kernel type.
        gamma=model.gamma if model.kernel in ['rbf', 'poly'] else 'scale',

        # This line of code also uses conditional logic.
        # The degree will be equal to 3 if the kernel type is not polynomial.
        degree=model.degree if model.kernel == 'poly' else 3,

        # This line of code assigns a balanced class weight.
        # This is appropriate for imbalanced data.
        class_weight='balanced'
    )

    # This line treats the new 2D SVM model using PCA-reduced training data.
    # This is necessary because the decision boundary depends on the input space.
    model_2D.fit(X_train_2D, y_train)

    # This line defines the horizontal range for the plot based on the first PCA component of test data.
    # It subtracts and adds 1 to the minimum and maximum to provide padding for visualization.
    x_min, x_max = X_test_2D[:, 0].min() - 1, X_test_2D[:, 0].max() + 1

    # This line defines the vertical range for the plot based on the second PCA component of test data.
    # The ranges are used to build the meshgrid later.
    y_min, y_max = X_test_2D[:, 1].min() - 1, X_test_2D[:, 1].max() + 1

    # This line creates a dense grid of coordinates covering the entire plot space with 500 points along each axis.
    # Each point in this grid will be used to predict class labels and color regions.
    xx, yy = np.meshgrid(np.linspace(x_min, x_max, 500), np.linspace(y_min, y_max, 500))

    # This line predicts class labels for every point in the meshgrid using the trained 2D model.
    # It reshapes the predictions into a matrix matching the grid's shape.
    Z = model_2D.predict(np.c_[xx.ravel(), yy.ravel()]).reshape(xx.shape)

    # This line defines a background color map for the decision regions.
    # The two colors represent the two predicted classes (low risk and high risk).
    cmap_background = ListedColormap(['#FFDDC1', '#C1D3FF'])

    # This line defines the color map for the actual data points.
    # The two colors contrast sharply with the background for clear separation.
    cmap_points = ListedColormap(['#D43F00', '#003F88'])

    # This line creates a new figure for the plot with a specified size.
    # A larger figure helps in rendering a detailed and readable chart.
    plt.figure(figsize=(8, 6))

    # This line draws the filled contour plot of decision regions using the prediction matrix Z.
    # The alpha value makes the background semi-transparent so that points are still visible.
    plt.contourf(xx, yy, Z, alpha=0.3, cmap=cmap_background)

    # This line overlays the test data points onto the contour plot.
    # It uses actual labels to color the points and adds a border for better visibility.

    # This line retrieves the handles (color patches) from the scatter plot.
    # These handles correspond to each class present in the test labels.
    handles, _ = scatter.legend_elements()

    # This line adds a legend to the plot explaining what each color of dot means.
    # It ensures clarity for anyone viewing the visualization.
    plt.legend(handles, ["Low Risk", "High Risk"], loc='upper right', title="True Labels")

    # This line sets the plot title using the model name provided to the function.
    # It highlights that the boundary shown is for PCA-reduced data.
    plt.title(f"SVM Decision Boundary - {model_name} (PCA Reduced)", fontsize=14, weight='bold')

    # This line labels the x-axis as the first principal component.
    # It helps us understand what the horizontal axis represents.
    plt.xlabel("Principal Component 1")

    # This line labels the y-axis as the second principal component.
    # It follows the same logic as labeling the x-axis.
    plt.ylabel("Principal Component 2")

    # This line adds a light dashed grid to the plot.
    # It helps in visually estimating the position of points and boundaries.
    plt.grid(True, linestyle='--', alpha=0.3)

    # This line adjusts plot elements to fit within the figure boundaries.
    # It prevents label and title clipping.
    plt.tight_layout()

    # This line renders the final plot.
    # It's the last step to display the decision boundary visualization.
    plt.show()
```

Figure N. Optimized Polynomial Kernel Evaluation

The function *visualize_svm_decision_boundary* was defined to aid in visualizing the decision boundary of each of the SVM Models. This applies the Principal Component Analysis (PCA) to reduce the training and testing datasets to two (2) principal components. This preserves the variance to capture a meaningful visualization. A new SVM model will be trained on the reduced training data with the same hyperparameters as the original model.

This function also generates a mesh grid of points across the 2D space and predicts its class labels to be mapped out on the decision boundaries. The regions are plotted with a color-coded background, with the test data points being on top.

Visualizing the Models

```
# This line calls the helper function to visualize the decision boundary of the optimized Linear SVM model.
# It passes the model along with the training and testing datasets to the function.
visualize_svm_decision_boundary(svm_linear_optimized, "Linear SVM Optimized", X_train, X_test, y_train, y_test)
```

```
# This line of code sends the RBF-optimized SVM model to the visualization function to see how it classifies in 2D PCA space.
# The function will retrain the model on PCA-reduced data and plot its decision boundaries.
visualize_svm_decision_boundary(svm_rbf_optimized, "RBF SVM Optimized", X_train, X_test, y_train, y_test)
```

```
# This line of code executes the same visualization process but with the Polynomial SVM model.
# It shows how the polynomial kernel's curved boundaries behave when reduced to 2D for interpretation.
visualize_svm_decision_boundary(svm_poly_optimized, "Polynomial SVM Optimized", X_train, X_test, y_train, y_test)
```

Figure N. Visualizing the Linear, RBF, and Polynomial Models

These code cells execute the function defined earlier to each of the optimized kernel models. The function, as mentioned earlier, will retrain the model on PCA-reduced data and plot its decision boundaries. This is why the training and the testing data is included in its parameters.

Visualization

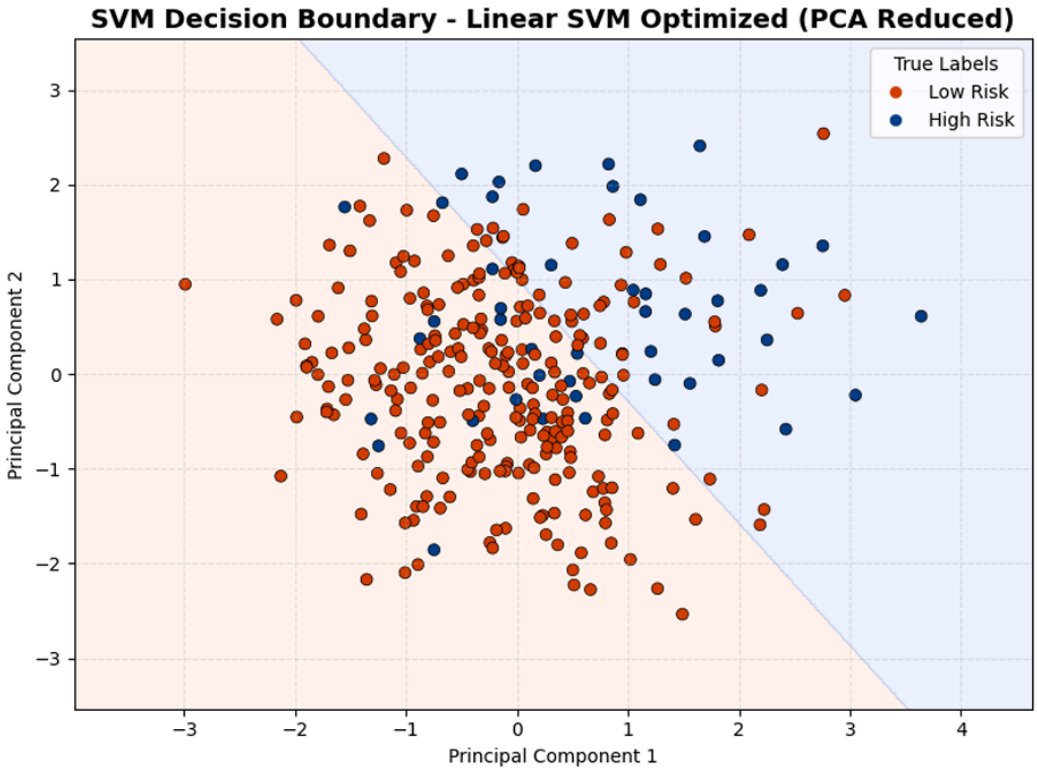


Figure N. Optimized Linear SVM Model Visualization

The visualization shows that the linear SVM model splits the classes along a diagonal. However, it is evident that blue high-risk points are still present in the low-risk decision boundary. This supports the evaluation earlier that this models performes well but needs further steps to perfectly separate classes.

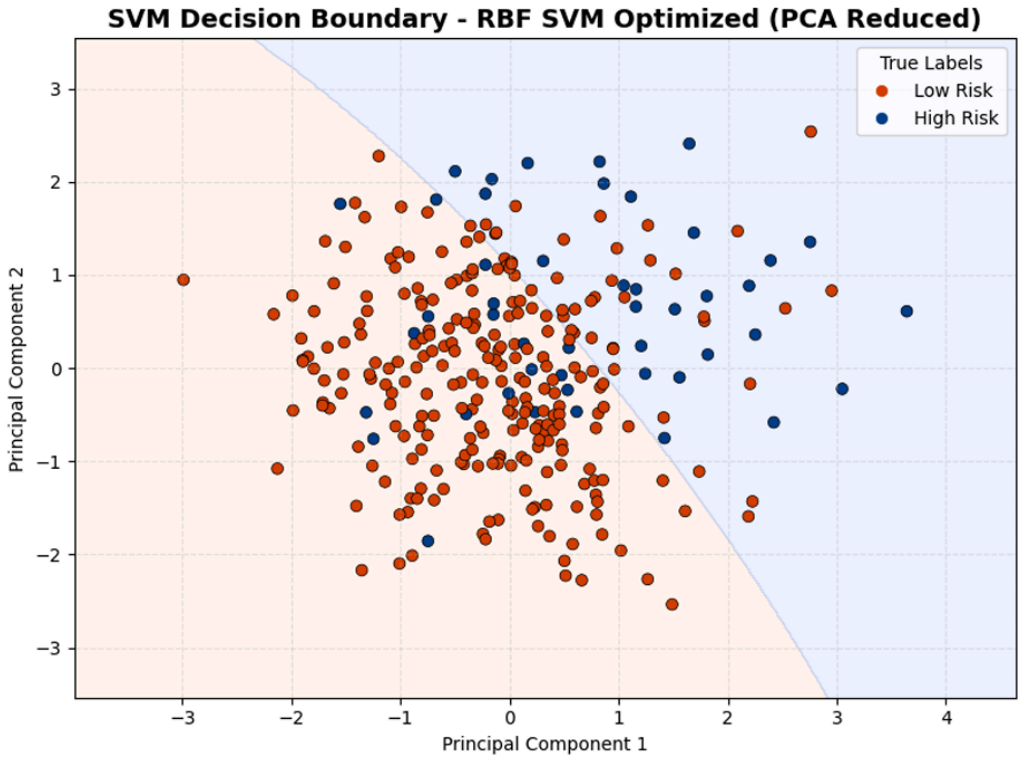


Figure N. Optimized RBF SVM Model Visualization

The visualization shows that the RBF SVM model creates a slightly curved boundary, highlighting the model's ability to be flexible as it is nonlinear. However, the curve is still similar to the linear boundary, which explains the similar results we have evaluated earlier for both optimized models. This confirms our metric results that RBF performed similarly to Linear SVM and that the dataset does not need more complex non-linear boundaries.

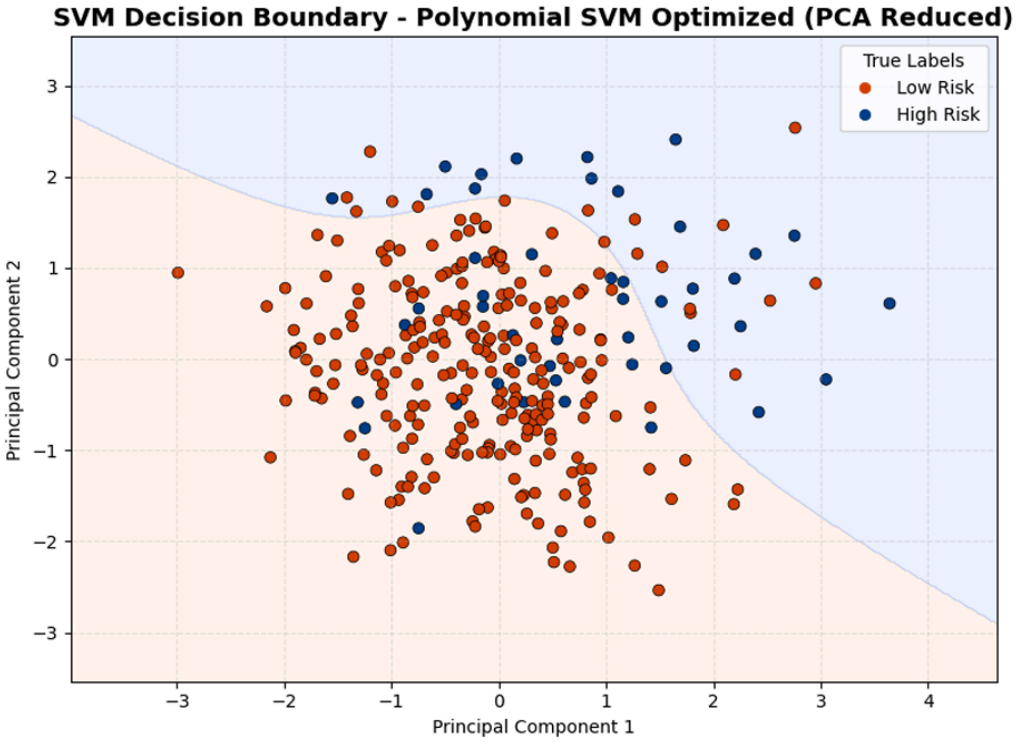


Figure N. Optimized Polynomial SVM Model Visualization

This visualization shows that the Polynomial kernel forms a more complex boundary with its evident wave or curve in the middle. This flexibility allows more adaption to data, however, the model still has many overlaps between classes. This aligns with our earlier evaluation that the Polynomial SVM did not help improve recall or precision, and more complexity may not be necessary for this dataset.

Overall Insights

The decision boundary visualizations here reinforce the observation that the RBF and Linear SVMs achieve very similar separation patterns, indicating that the structure of the dataset is very close to being linearly separable even after dimensionality reduction. The Polynomial SVM constructs a more complicated boundary, but does not make distinctly better contributions to separate the two classes, as would be in line with its slightly lower recall and F1-score in the previous comparisons. Overall, the visualizations support that simpler models, such as the Linear SV,M are adequate for this dataset since model complexity does not significantly enhance classification performance.

Support Vector Machine Analysis:
Objective Points Discussion

Part 1: SVM Classification Effectiveness

How well different SVM configurations (Linear, RBF, Polynomial) can classify individuals into "High Risk" or "Low Risk" categories based on the provided features.

The three SVM configurations, Linear, RBF, and Polynomial, have **consistently shown high overall classification accurac**y, ranging around **90%-91%** in accurately classifying the low-risk and high-risk categories. However, this high accuracy is **influenced by a class imbalance** where majority of the patients are low-risk and only around 150 patients are labeled as high-risk. This issue is present in the confusion matrices, where all three models **performed well on the Low Risk class (with precision ≈ 0.93 and recall ≈ 0.96)**, while only a **moderate performance is observable on the High Risk class (recall ≈ 0.63 for Linear and RBF and ≈ 0.61 for Polynomial)**. The same are still observable despite hypertuning. Overall, the SVM models are very effective for predicting low-risk patients and only have a moderate effectiveness for high-risk individuals. While this may be a good model performance, real-world scenarios regarding health are very critical and require a higher accurate model.

Part 2: Impact of Features

The significance of different health attributes (features) in the heart attack risk prediction task. (Though this dataset is synthetic, infer general importance).

The data exploration part of this activity has revealed ***Diabetes*** to be the number one feature attributed to high risks, followed by ***Smoking***. This conclusion has been supported by both correlational analyses, where *Diabetes* had Pearson correlation of **~0.63** and *Smoking* **~0.31** with *HeartAttackRisk*. The ANOVA F-test also showed that same top features, rendering both features as statistically significant predictors.

On the other hand, other features, including *Age*, *Cholesterol*, *SystolicBP*, *BMI*, and *ExerciseHours* showed **weaker correlations** and **insignificant ANOVA results**. While this indicates that they are not strong predictors as individual features, this suggests that they could contribute in combination with other features.

Part 3: The Critical Role of Kernel Functions

How the choice of kernel (Linear, RBF, Polynomial) fundamentally changes the decision boundary and the model's ability to handle linear vs. non-linear patterns in the data.

The kernel functions in SVM help in identifying how the model would be separating classes: linear, nonlinear, or a polynomial curve. The Linear SVM decision boundary in this data has successfully separated classes with a straight diagonal line in the PCA space, with a few overlaps. The RBF SVM decision boundary showed a slight curve similar to the Linear SVM. The Polynomial

SVM has a more flexible and wavy decision boundary that tries to adapt to the data, but is worsened with a decreased recall and more overlaps.

The SVM visualizations show that increasing the kernel's complexity for linearly classifiable data does not improve classification. The Linear and RBF models, which are simpler, handle this data better.

Part 4: The Impact of Hyperparameters

Hyperparameters have been crucial in balancing the model's complexity and generalization. The Linear SVM's best **C = 0.01** indicated that a simpler model with strong regularization was optimal to avoid overfitting. For the RBF kernel, the best configuration was **C = 1, gamma = 0.01,** which suggests a moderate decision boundary smoothness and balance between bias and variance. The Polynomial kernel's optimal parameters were **C = 1, degree = 3, gamma = scale**, and showed the need for a moderately flexible model, but its performance did not do better than the simpler models

This confirms that hyperparameters like C control the margin's flexibility, gamma controls the tightness of model fitting around data points, and degree, for polynomials, adjusts boundary complexity.

Part 5: Importance of Data Preprocessing

Data Preprocessing has always been very crucial in modeling. This activity highlighted the importance of feature scaling that was required for SVM models as they are reliant on distance-based calculation between points and hyperplanes. Standardizing the data has helped ensure that no feature has biasedly influenced the model due to different scales. Furthermore, the data splitting for training and testing (70/30) has helped us observe different model behavior and accuracy. Our previous split was (80/20), which resulted in the same accuracies for each model and was not as insightful as this split. Overall, data preprocessing is very essential for an unbiased evaluation and an insightful data modeling.

Part 6: Interpreting Evaluation Metrics in Context

For the heart attack risk prediction context, recall is critical for the high-risk class. In the real world, missing a high-risk patient (false negative) could present critical health implications. A misclassification on a low-risk patient as high-risk only adds an extra precaution for the patients. he models showed a recall of ~0.63 for high-risk patients, meaning they correctly identified 63% but missed 37% of actual high-risk individuals. Due to this model involving healthcare, the priority must rely on minimizing false negatives, even at the cost of reducing precision. Overall, despite a strong accuracy, the model's performance with a moderate recall needs further improvement.

Part 7: Visualizing Decision Boundaries

Visualizing the decision boundaries after PCA reduction provided clear insight into how each kernel attempts to classify data:

- The Linear SVM produced a straight-line separation.
    - This effectively divides most points, but with still a few significant overlaps.
-
- The RBF SVM produced a slightly curved boundary, similar to the Linear SVM
    - This showed its capacity for non-linear classification
    - This performance is very similar to the linear model.
-
- The Polynomial SVM produced a visibly wavy boundary
    - This demonstrated a greater flexibility, but also captured noise.
    - This resulted in a slight drop in performance.

These visualizations helped the evaluation metrics become more insightful and meaningful. This includes the fact that despite kernel complexity, the dataset allowed linear classifiers to perform sufficiently, and increasing model complexity only worsened the decision boundary and performance.